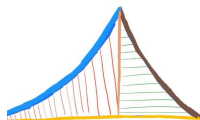


# RGB to Black and White: A Clear Journey

---



G. V. V. Sharma

## ABOUT THIS ARTICLE

This article is written by T. P. Siddardha Reddy under the guidance of Dr. GVV Sharma, an Associate Professor in IITH EE Department.

This article presents a clear journey from a coloured RGB image to a black-and-white representation. The objective is to understand the representation of RGB image data, the storage of raw bitmap data, the role of colour conversion, and the reduction of colour information to intensity information.

The presentation follows the style used throughout Dr GVV Sharma's documents: definitions and formulae are introduced first, followed by the conversion procedure, clustering methods, implementation, and output. The mathematical expressions are kept explicit so that the complete conversion can be followed from the original RGB values to the final black-and-white image.

Github: <https://github.com/gadepall/>

License: <https://creativecommons.org/licenses/by-sa/3.0/>

Published on: 3rd October 2026

## CONTENTS

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                       | <b>5</b>  |
| 1.1      | RGB Image . . . . .                                       | 5         |
| <b>2</b> | <b>RGB Image Representation</b>                           | <b>5</b>  |
| 2.1      | RGB Pixel Representation . . . . .                        | 5         |
| 2.2      | Colour Information . . . . .                              | 5         |
| <b>3</b> | <b>Raw Bitmap and JPEG Representation</b>                 | <b>6</b>  |
| 3.1      | Raw Bitmap Representation . . . . .                       | 6         |
| 3.2      | JPEG Representation . . . . .                             | 6         |
| 3.3      | Per-Pixel Storage . . . . .                               | 6         |
| <b>4</b> | <b>Colour Conversion</b>                                  | <b>7</b>  |
| 4.1      | RGB to Luminance and Chrominance . . . . .                | 7         |
| 4.2      | Chrominance Components . . . . .                          | 7         |
| 4.3      | Luminance Weighting . . . . .                             | 8         |
| 4.4      | Information Reduction . . . . .                           | 8         |
| <b>5</b> | <b>RGB to Grayscale Conversion</b>                        | <b>8</b>  |
| 5.1      | Grayscale Intensity . . . . .                             | 8         |
| 5.2      | Black and White . . . . .                                 | 8         |
| <b>6</b> | <b>Grayscale and Black-and-White Conversion Algorithm</b> | <b>9</b>  |
| 6.1      | RGB to Grayscale Algorithm . . . . .                      | 9         |
| 6.2      | Mapping to the Output Image . . . . .                     | 9         |
| <b>7</b> | <b>Clustering Methods and Analysis</b>                    | <b>9</b>  |
| 7.1      | Global Mean Thresholding . . . . .                        | 9         |
| 7.2      | Static Reference Thresholding . . . . .                   | 10        |
| 7.3      | Gaussian-Weighted Local Reference . . . . .               | 10        |
| 7.4      | Importance of $\sigma$ . . . . .                          | 11        |
| 7.5      | Purpose of $C = 3$ . . . . .                              | 11        |
| 7.6      | Gaussian Local Thresholding . . . . .                     | 11        |
| 7.7      | Computational Consideration . . . . .                     | 11        |
| <b>8</b> | <b>Implementation and Output</b>                          | <b>12</b> |
| 8.1      | Implementation . . . . .                                  | 12        |
| 8.2      | Testing Case 1: Scenery . . . . .                         | 12        |
| 8.3      | Testing Case 2: Jersey . . . . .                          | 15        |
| 8.4      | Program File . . . . .                                    | 16        |

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>9</b> | <b>References</b>                               | <b>17</b> |
| 9.1      | Reference 1: RGB Image Representation . . . . . | 17        |
| 9.2      | Reference 2: JPEG Image Compression . . . . .   | 17        |
| 9.3      | Reference 3: Y, Cb and Cr Conversion . . . . .  | 17        |
| 9.4      | Reference 4: Image Thresholding . . . . .       | 17        |
| 9.5      | Reference 5: Gaussian Filtering . . . . .       | 17        |

## 1. INTRODUCTION

### 1.1. RGB Image

An RGB image represents colour using three components: Red, Green, and Blue. A pixel can therefore be represented by the tuple

$$P_{ij} = (R_{ij}, G_{ij}, B_{ij}) \quad (1.1.1)$$

For a conventional 8-bit RGB image, each component is represented using 8 bits. Hence, one pixel requires

$$8 + 8 + 8 = 24 \quad (1.1.2)$$

bits of colour information.

For an image containing  $M \times N$  pixels, the image may be viewed as three matrices,

$$R \in \mathbb{R}^{M \times N} \quad (1.1.3)$$

$$G \in \mathbb{R}^{M \times N} \quad (1.1.4)$$

$$B \in \mathbb{R}^{M \times N} \quad (1.1.5)$$

The conversion to black and white is therefore a reduction from three colour values per pixel to one intensity value per pixel.

## 2. RGB IMAGE REPRESENTATION

### 2.1. RGB Pixel Representation

The three components of an RGB pixel are normally represented in the range 0 to 255 for an 8-bit image:

$$0 \leq R, G, B \leq 255 \quad (2.1.1)$$

The pixel at position  $(i, j)$  is

$$P_{ij} = (R_{ij}, G_{ij}, B_{ij}) \quad (2.1.2)$$

### 2.2. Colour Information

Different RGB tuples may produce colours having similar perceived brightness. Consequently, choosing only one RGB component would discard important information. A

weighted combination is therefore used when the objective is to obtain grayscale intensity.

### 3. RAW BITMAP AND JPEG REPRESENTATION

#### 3.1. Raw Bitmap Representation

A raw RGB bitmap stores the colour components of the pixels directly. For a  $2 \times 2$  group of pixels,

$$\begin{bmatrix} (R_{11}, G_{11}, B_{11}) & (R_{12}, G_{12}, B_{12}) \\ (R_{21}, G_{21}, B_{21}) & (R_{22}, G_{22}, B_{22}) \end{bmatrix} \quad (3.1.1)$$

#### 3.2. JPEG Representation

JPEG uses a transformed and compressed representation rather than storing every RGB tuple independently. The JPEG standard defines processes for converting source image data to compressed image data.

The common colour information is calculated, and the formulae used for this conversion are listed in the next section.

#### 3.3. Per-Pixel Storage

For an image containing  $N$  pixels, an 8-bit RGB representation requires three separate 8-bit colour components for every pixel. Hence, the raw colour information requires

$$3 \times 8N = 24N \quad (3.3.1)$$

bits.

If the image is converted directly from RGB to grayscale, only the luminance value  $Y$  is retained for each pixel. With an 8-bit grayscale representation, this requires

$$8N = 8N \quad (3.3.2)$$

bits.

Thus, the storage changes from  $24N$  bits for RGB to  $8N$  bits for grayscale. The storage ratio is therefore

$$\frac{8N}{24N} = \frac{1}{3} \quad (3.3.3)$$

so the grayscale representation requires one-third of the raw RGB storage, corresponding to a reduction of two-thirds.

When the image is represented using  $Y$ ,  $Cb$ , and  $Cr$ , the three components are still separate components. Each component is normally represented using 8-bit samples; the

reduction in storage can instead be obtained by reducing the spatial sampling of the chrominance components. In particular, with 4:2:0 chroma subsampling, the  $Y$  component retains  $N$  samples, while  $Cb$  and  $Cr$  each contain  $N/4$  samples.

Therefore, before the additional JPEG transform and coding stages, the sample storage can be written as

$$8N + 8\frac{N}{4} + 8\frac{N}{4} = 12N \quad (3.3.4)$$

bits.

Thus, the factor of four applies to the *number of chrominance samples*, not to the number of bits in an individual  $Cb$  or  $Cr$  sample. The  $Y$  component is retained at full spatial resolution because it carries the intensity information for every pixel.

For example, in a  $2 \times 2$  group containing four  $Y$  samples, 4:2:0 subsampling uses one  $Cb$  sample and one  $Cr$  sample for the group. The  $Cb$  and  $Cr$  samples are therefore stored separately, but they are shared spatially by the four corresponding pixels.

The comparison above describes sample storage before the main JPEG compression stages. The actual size of a JPEG file is different because JPEG additionally uses transformation, quantisation, and entropy coding as part of its coding process.

## 4. COLOUR CONVERSION

### 4.1. RGB to Luminance and Chrominance

The RGB components may be transformed into three separate components, luminance  $Y$  and chrominance  $Cb$  and  $Cr$ :

$$Y = 0.299R + 0.587G + 0.114B \quad (4.1.1)$$

$$Cb = -0.168736R - 0.331264G + 0.5B + 128 \quad (4.1.2)$$

$$Cr = 0.5R - 0.418688G - 0.081312B + 128 \quad (4.1.3)$$

Here,  $Y$  represents luminance, while  $Cb$  and  $Cr$  represent chrominance information. The  $Cb$  and  $Cr$  components carry colour-difference information associated with blue and red, respectively. They are separate 8-bit components in the usual 8-bit representation; they are not single-bit colour indicators.

### 4.2. Chrominance Components

The  $Cb$  component describes the blue-related colour difference and  $Cr$  describes the red-related colour difference. These components retain colour information that is not required when only grayscale intensity is required.

When chroma subsampling is used, the  $Cb$  and  $Cr$  components can be stored at a lower spatial sampling rate than  $Y$ . For example, in 4:2:0 subsampling, each  $2 \times 2$  group

of pixels retains four  $Y$  samples but uses one  $Cb$  sample and one  $Cr$  sample. Thus, the chrominance components remain separate components, while fewer chrominance samples are stored.

#### 4.3. Luminance Weighting

The coefficients in the luminance equation are different because the human visual system does not perceive the three RGB components with equal sensitivity:

$$0.587 > 0.299 > 0.114 \quad (4.3.1)$$

The green component consequently has the largest contribution to luminance, followed by red and then blue.

#### 4.4. Information Reduction

For the purpose of grayscale conversion, the colour pixel  $(R, G, B)$  is reduced to the single luminance value  $Y$ . The  $Cb$  and  $Cr$  components carry the colour-difference information and are therefore not required for the final grayscale intensity image.

## 5. RGB TO GRAYSCALE CONVERSION

### 5.1. Grayscale Intensity

A grayscale image represents every pixel using a single intensity value:

$$Y = 0.299R + 0.587G + 0.114B \quad (5.1.1)$$

A small value of  $Y$  represents a dark pixel, while a large value represents a bright pixel.

### 5.2. Black and White

A grayscale image contains many intensity levels between black and white. A black-and-white image instead contains two classes.

Let  $T$  denote the selected threshold:

$$B(i, j) = \begin{cases} 0, & Y(i, j) < T, \\ 255, & Y(i, j) \geq T. \end{cases} \quad (5.2.1)$$

Thus the grayscale intensity is mapped to one of two output values.

## 6. GRAYSCALE AND BLACK-AND-WHITE CONVERSION ALGORITHM

### 6.1. RGB to Grayscale Algorithm

The conversion procedure is carried out pixel by pixel.

- 1) Read the RGB input image.
- 2) Obtain  $R$ ,  $G$ , and  $B$  for each pixel.
- 3) Calculate the luminance  $Y$ .
- 4) Store  $Y$  as the grayscale intensity.
- 5) Select the threshold  $T$ .
- 6) Compare every grayscale value with  $T$ .
- 7) Assign black or white according to the threshold condition.
- 8) Save the resulting image.

The complete transformation is

$$RGB \rightarrow Y \rightarrow \text{Threshold} \rightarrow \text{Black/White} \quad (6.1.1)$$

### 6.2. Mapping to the Output Image

The calculated intensity matrix is passed to the image-writing stage. When JPEG is selected, the image-writing stage performs the corresponding JPEG encoding and compression.

## 7. CLUSTERING METHODS AND ANALYSIS

### 7.1. Global Mean Thresholding

The first dynamic method calculates one reference intensity from the complete grayscale image. If the grayscale matrix is denoted by  $Y$ , the reference is the mean intensity of all pixels:

$$\mu = \frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N Y(i, j) \quad (7.1.1)$$

Every pixel is then compared with this single image-wide reference. The corresponding binary decision is

$$B(i, j) = \begin{cases} 255, & Y(i, j) \geq \mu, \\ 0, & Y(i, j) < \mu. \end{cases} \quad (7.1.2)$$

This method is adaptive in the sense that the reference changes with the input image, but the reference itself is global. It therefore does not change from one location to another within the same image.

### 7.2. Static Reference Thresholding

For comparison, a fixed reference of 128 is used in the static thresholding method. A grayscale pixel is assigned to white when its intensity is at least 128 and to black otherwise:

$$B(i, j) = \begin{cases} 255, & Y(i, j) \geq 128, \\ 0, & Y(i, j) < 128. \end{cases} \quad (7.2.1)$$

The purpose of this fixed reference is to provide a simple baseline against which the dynamically calculated references can be compared.

### 7.3. Gaussian-Weighted Local Reference

A more localized reference is obtained by applying a Gaussian filter to the grayscale intensity matrix. Instead of giving every neighbouring pixel equal importance, the Gaussian function gives greater importance to pixels close to the point being considered and progressively less importance to pixels farther away.

The Gaussian weighting function is

$$w(d) = \exp\left(-\frac{d^2}{2\sigma^2}\right) \quad (7.3.1)$$

Here,  $d$  is the distance from the point being processed and  $\sigma$  controls the spatial spread of the Gaussian weighting. The corresponding local reference can be understood as the weighted average

$$\mu_G(i, j) = \frac{\sum_{(u,v) \in \mathcal{N}} w(d_{uv})Y(u, v)}{\sum_{(u,v) \in \mathcal{N}} w(d_{uv})} \quad (7.3.2)$$

Thus, the reference intensity is different at different locations in the image and follows the local intensity distribution smoothly.

In the implementation, this operation is performed using `gaussian_filter`:

```
local_mean = gaussian_filter(
    grayscale_matrix.astype(np.float32), sigma=sigma
)
```

#### 7.4. Importance of $\sigma$

The value of  $\sigma$  determines how localized the Gaussian reference is. A smaller value concentrates the weighting around the current pixel, so the reference responds strongly to nearby intensity variations. A larger value spreads the weighting over a wider region, causing pixels farther from the current point to contribute more significantly.

Consequently,  $\sigma = 10$  gives the localized reference used in this implementation, whereas  $\sigma = 20$  gives the wider reference. As  $\sigma$  becomes sufficiently large, the Gaussian weights over the relevant region become increasingly similar. The Gaussian-weighted average therefore approaches the behaviour of an ordinary average over a broad neighbourhood.

#### 7.5. Purpose of $C = 3$

After the Gaussian local reference is calculated, the implementation uses the threshold

$$T_G(i, j) = \mu_G(i, j) - C \quad (7.5.1)$$

The constant  $C = 3$  is introduced as a small offset to the Gaussian local reference. Subtracting this value slightly lowers the local threshold, allowing pixels that are marginally darker than the calculated local mean to still be classified as white. In this implementation,  $C = 3$  is used as a small empirical adjustment to fine-tune the sensitivity of the resulting black-and-white output.

#### 7.6. Gaussian Local Thresholding

The complete decision made by the Gaussian method is therefore

$$B(i, j) = \begin{cases} 255, & Y(i, j) \geq \mu_G(i, j) - 3, \\ 0, & Y(i, j) < \mu_G(i, j) - 3. \end{cases} \quad (7.6.1)$$

The important difference from the global methods is that  $\mu_G(i, j)$  is not one constant for the entire image. It is a spatially varying reference calculated from the surrounding grayscale intensities.

#### 7.7. Computational Consideration

The Gaussian method provides a locally varying reference, but this comes at an increased computational cost compared with a single global mean. Conceptually, the local

reference has to be evaluated throughout the image by considering the surrounding pixels and their Gaussian contributions. The process can therefore be viewed as

$$\text{pixel} \rightarrow \text{neighbourhood} \rightarrow \text{Gaussian weights} \rightarrow \text{local reference} \rightarrow \text{threshold} \quad (7.7.1)$$

A larger effective Gaussian neighbourhood also involves a broader set of surrounding pixels. Thus, although the Gaussian reference provides better spatial adaptation, a direct pixel-by-pixel implementation can be time-consuming. The `scipy.ndimage.gaussian_filter` function used here performs the filtering efficiently while implementing the same Gaussian smoothing principle.

## 8. IMPLEMENTATION AND OUTPUT

### 8.1. Implementation

The implementation reads an RGB JPEG image and converts the RGB matrix to a YCbCr representation. The Y component is then retained as the grayscale intensity matrix. From this grayscale matrix, three different reference strategies are tested: a fixed reference of 128, a global mean reference, and Gaussian local references with two different values of  $\sigma$ .

The purpose of taking two different scenarios is to clearly illustrate that the mode of clustering and averaging chosen depends on the type of image. For Example images with simple graphic arts prefer gaussian but those with complex graphics or natural photos work well with the global average referencing.

Hence we have to use some machine learning algorithms and models so they can be trained with the data available about best referencing based on the situation, which will be used by them further for a new situation, and this implementation is beyond the scope of this article.

### 8.2. Testing Case 1: Scenery

The first test case uses a scenery image. The outputs are presented in the following order so that the effect of each reference method can be compared with the original image.

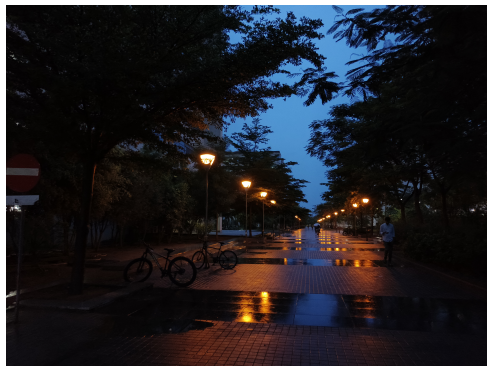


Fig. 8.1. Input scenery image.



Fig. 8.2. Grayscale representation of the scenery image.



Fig. 8.3. Global mean adaptive thresholding of the scenery image.

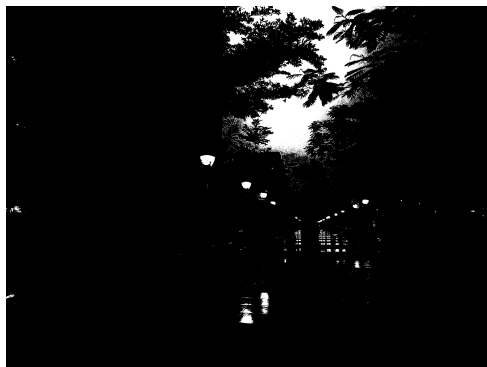


Fig. 8.4. Static thresholding using the fixed reference of 128.

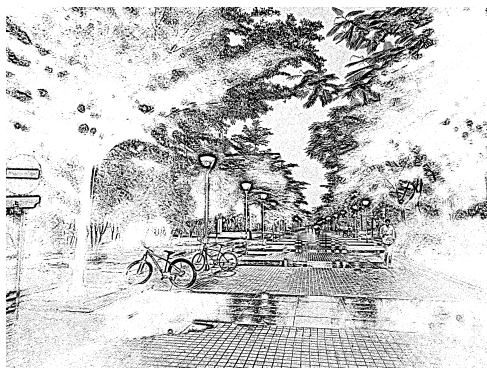


Fig. 8.5. Gaussian localized reference with  $\sigma = 10$  and  $C = 3$ .



Fig. 8.6. Gaussian wide reference with  $\sigma = 1200$  and  $C = 3$ .

### 8.3. Testing Case 2: Jersey

The second test case uses a jersey image. The same sequence is retained to make the behaviour of the reference methods directly comparable.



Fig. 8.7. Input jersey image.



Fig. 8.8. Grayscale representation of the jersey image.



Fig. 8.9. Global mean adaptive thresholding of the jersey image.



## 9. REFERENCES

### 9.1. Reference 1: RGB Image Representation

ITU-T Recommendation T.871, *JPEG File Interchange Format (JFIF)*. <https://www.itu.int/rec/T-REC-T.871/en>

### 9.2. Reference 2: JPEG Image Compression

ITU-T Recommendation T.81, *Information technology – Digital compression and coding of continuous-tone still images – Requirements and guidelines*. <https://www.itu.int/rec/T-REC-T.81/en>

### 9.3. Reference 3: Y, Cb and Cr Conversion

ITU-R Recommendation BT.601, *Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios*. <https://www.itu.int/rec/R-REC-BT.601/en>

### 9.4. Reference 4: Image Thresholding

scikit-image Documentation, *Image Thresholding*. [https://scikit-image.org/docs/stable/auto\\_examples/applications/plot\\_thresholding\\_guide.html](https://scikit-image.org/docs/stable/auto_examples/applications/plot_thresholding_guide.html)

### 9.5. Reference 5: Gaussian Filtering

SciPy Documentation, *Multidimensional Gaussian Filter*. [https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.gaussian\\_filter.html](https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.gaussian_filter.html)